

TECHNICAL DEPLOYMENT GUIDE

VPS Deployment Guide for the RCSI Dashboard

Step-by-step instructions for production
deployment

This guide covers everything needed to deploy the El Salvador
Division RCSI Dashboard to a Virtual Private Server (VPS): server
setup, application deployment, web server and SSL configuration,
post-deployment tasks, maintenance, and troubleshooting.

VERSION	1.0
AUDIENCE	IT Administrators
ISSUED	2026 · El Salvador Division

Table of Contents

Chapter 1 — Introduction & Overview3

1.1 What you will need3

1.2 Recommended VPS providers3

Chapter 2 — Server Preparation4

2.1 Connect to your VPS via SSH4

2.2 Update the system4

2.3 Install Node.js 20 LTS4

2.4 Install Bun (package manager)4

2.5 Install PM2 (process manager)4

Chapter 3 — Deploy the Application6

3.1 Create the application directory6

3.2 Transfer the application files6

3.3 Install dependencies on the server6

3.4 Set up the database6

3.5 Build the production bundle7

3.6 Start the app with PM27

Chapter 4 — Web Server & SSL Configuration8

4.1 Option A: Caddy (recommended — easiest)8

4.2 Option B: nginx (alternative)8

Chapter 5 — Post-Deployment Configuration10

5.1 Remove the "Prototype — For Approval" badge10

5.2 Add the official El Salvador Division logo10

5.3 Run clean slate for pilot deployment	10
5.4 Set up automatic database backups	10
Chapter 6 — Maintenance & Monitoring	12
6.1 Daily monitoring commands	12
6.2 Updating the application	12
6.3 Restarting after a server reboot	12
6.4 Viewing the upload audit log	12
Chapter 7 — Troubleshooting	14
7.1 Getting help	14
Appendix A — Quick Reference Card	15
A.1 Essential Commands	15
A.2 File Locations	15

Chapter 1 — Introduction & Overview

This guide provides step-by-step instructions for deploying the El Salvador Division Research Culture Sustainability Index (RCSI) Dashboard to a Virtual Private Server (VPS). A VPS deployment is recommended for production use because it provides a permanent, stable hosting environment with full control over the server configuration, database, and security.

Unlike serverless cloud platforms (which have size limits and compatibility issues with SQLite databases), a VPS runs the application as a traditional Node.js server process. This means the SQLite database file persists on disk, native binaries run without restrictions, and the application can handle concurrent uploads from multiple schools simultaneously.

1.1 What you will need

Requirement	Specification	Estimated Cost
VPS Server	Ubuntu 22.04 or 24.04 LTS, 2+ CPU cores, 4 GB RAM, 40 GB SSD	\$5–\$20/month
Domain name	Optional but recommended (e.g., rcsi.elsalvadordivision.ph)	\$10–\$15/year
SSL Certificate	Free via Let's Encrypt (automated with Caddy or nginx)	Free
SSH access	Root or sudo user access to the VPS	—
Local computer	With SSH client (Terminal on Mac/Linux, PuTTY on Windows)	—

Table 1.1 — Deployment requirements and estimated costs.

1.2 Recommended VPS providers

Any VPS provider that offers Ubuntu LTS will work. Here are three popular options that the Division may consider, listed from most affordable to most feature-rich:

Provider	Plan	Price	Notes
DigitalOcean	Basic Droplet (2 CPU, 4 GB RAM)	\$24/month	Simplest setup, excellent documentation, Manila data center available
Hetzner	CX22 (2 CPU, 4 GB RAM)	€4.50/month	Best value, European data centers, very reliable
AWS EC2	t3.medium (2 CPU, 4 GB RAM)	~\$30/month	Most features, complex billing, Singapore region available

Table 1.2 — Recommended VPS providers.

Using an existing server

If the Division has an existing server (e.g., a DepEd-provided machine or a local computer that is always on), you can use that instead of renting a VPS. The deployment steps are the same — just skip the "rent a VPS" step and SSH into your existing machine.

Chapter 2 — Server Preparation

This chapter covers the initial setup of your VPS: connecting to it, updating the system, and installing the required software (Node.js, Bun, and PM2). All commands in this guide are run in the terminal (SSH) and assume Ubuntu 22.04 or 24.04 LTS.

2.1 Connect to your VPS via SSH

After renting a VPS, you will receive an IP address and root password. Open a terminal on your local computer and connect:

```
# On Mac/Linux Terminal or Windows PowerShell:
ssh root@YOUR_SERVER_IP

# Enter the password when prompted
# You should now see a prompt like: root@server:~#
```

2.2 Update the system

Run these commands to update all system packages to the latest versions:

```
apt update && apt upgrade -y
apt install -y curl git build-essential python3
```

2.3 Install Node.js 20 LTS

The RCSI Dashboard runs on Next.js 16, which requires Node.js 20 or later. Install it using the NodeSource repository:

```
curl -fsSL https://deb.nodesource.com/setup_20.x | bash -
apt install -y nodejs

# Verify installation:
node --version # Should show v20.x.x
npm --version # Should show 10.x.x
```

2.4 Install Bun (package manager)

The project uses Bun as its package manager. Install it:

```
curl -fsSL https://bun.sh/install | bash

# Add Bun to PATH (run this, then close and reopen the terminal):
echo 'export PATH="$HOME/.bun/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc

# Verify:
bun --version # Should show 1.x.x
```

2.5 Install PM2 (process manager)

PM2 keeps the application running permanently, even after the server reboots. It also provides logging and automatic restarts if the app crashes.

```
npm install -g pm2

# Verify:
pm2 --version # Should show 5.x.x
```

Why PM2?

PM2 is the key to keeping the app online 24/7. Without it, the app would stop running if the SSH session closes or the server reboots. PM2 starts the app automatically on boot and restarts it if it crashes for any reason.

Chapter 3 — Deploy the Application

This chapter covers transferring the application files to the server, installing dependencies, building the production bundle, and starting the app with PM2.

3.1 Create the application directory

```
mkdir -p /var/www/rcsi-dashboard
cd /var/www/rcsi-dashboard
```

3.2 Transfer the application files

From your LOCAL computer (not the server), upload the project files using SCP or rsync. Replace YOUR_SERVER_IP with the actual IP address:

```
# Option A: Using SCP (from your local computer)
scp -r /path/to/rcsi-dashboard/* root@YOUR_SERVER_IP:/var/www/rcsi-dashboard/

# Option B: Using rsync (faster, skips unchanged files)
rsync -avz --exclude node_modules --exclude .next \
/path/to/rcsi-dashboard/ root@YOUR_SERVER_IP:/var/www/rcsi-dashboard/
```

Important: Exclude node_modules

Exclude **node_modules** and **.next** directories when uploading — they will be regenerated on the server. Uploading them would take much longer (1.3 GB) and may cause compatibility issues.

3.3 Install dependencies on the server

Back on the SERVER (via SSH), install the Node.js dependencies:

```
cd /var/www/rcsi-dashboard
bun install

# This takes 2–5 minutes. Wait for it to complete.
```

3.4 Set up the database

The app uses a SQLite database. The database file should be in the db/ directory. Create it and set the environment variable:

```
# Create the db directory if it doesn't exist
mkdir -p db

# Create the .env file with the database path
echo 'DATABASE_URL="file:./db/custom.db"' > .env

# Push the database schema
bun run db:push

# (Optional) Seed with demo data for testing
# bun run scripts/seed.ts

# OR start with a clean slate (recommended for pilot)
# bun run scripts/clean-slate.ts
```

3.5 Build the production bundle

Build the optimized production version of the app:

```
bun run build

# This takes 30–60 seconds. You should see:
# ✓ Compiled successfully
# ✓ Generating static pages (12/12)
```

3.6 Start the app with PM2

Start the production server using PM2:

```
# Start the app
pm2 start "bun run start" --name rcsi-dashboard

# Save PM2 process list (so it restarts on reboot)
pm2 save

# Enable PM2 to start on boot
pm2 startup
# (Follow the instructions PM2 prints, which usually means
# copying and pasting one command)
```

The app is now running on port 3000. Test it by running:

```
curl http://localhost:3000/api/overview

# You should see JSON data with schoolCount, kpis, etc.
# If you see data, the app is working!
```

Troubleshooting

If the command returns an error, check the PM2 logs: `pm2 logs rcsi-dashboard --lines 50` Common issues: database path not found (check `.env`), port already in use (change the port in `package.json`), or missing dependencies (re-run `bun install`).

Chapter 4 — Web Server & SSL Configuration

The app runs on port 3000, but users need to access it via a standard web URL (port 80 for HTTP, port 443 for HTTPS). This chapter covers setting up a reverse proxy with Caddy (the easiest option) or nginx, plus free SSL certificates via Let's Encrypt.

4.1 Option A: Caddy (recommended — easiest)

Caddy is the simplest reverse proxy. It automatically obtains and renews SSL certificates from Let's Encrypt — no manual configuration needed.

```
# Install Caddy
apt install -y debian-keyring debian-archive-keyring apt-transport-https
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/gpg.key' | \
gpg --dearmor -o /usr/share/keyrings/caddy-stable-archive-keyring.gpg
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt' | \
tee /etc/apt/sources.list.d/caddy-stable.list
apt update
apt install -y caddy
```

Edit the Caddy configuration file:

```
nano /etc/caddy/Caddyfile

# Replace the contents with:
rcsi.elsalvadordivision.ph {
reverse_proxy localhost:3000
}

# Save (Ctrl+O, Enter) and exit (Ctrl+X)

# Restart Caddy
systemctl restart caddy

# Caddy will automatically obtain an SSL certificate
# Check status:
systemctl status caddy
```

Domain configuration

Replace **rcsi.elsalvadordivision.ph** with your actual domain name. If you don't have a domain yet, you can use the server's IP address temporarily (SSL won't work without a domain, but HTTP will).

4.2 Option B: nginx (alternative)

If you prefer nginx (or the Division already uses it), here is the configuration:

```
# Install nginx and Certbot
apt install -y nginx certbot python3-certbot-nginx

# Create the site configuration
nano /etc/nginx/sites-available/rcsi-dashboard

# Paste this configuration:
server {
listen 80;
```

```
server_name rcsi.elsalvadordivision.ph;

location / {
    proxy_pass http://localhost:3000;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection 'upgrade';
    proxy_set_header Host $host;
    proxy_cache_bypass $http_upgrade;
}

# Enable the site
ln -s /etc/nginx/sites-available/rcsi-dashboard /etc/nginx/sites-enabled/
nginx -t && systemctl reload nginx

# Obtain SSL certificate
certbot --nginx -d rcsi.elsalvadordivision.ph
```

Chapter 5 — Post-Deployment Configuration

After the app is deployed and accessible via HTTPS, complete these final configuration steps.

5.1 Remove the "Prototype — For Approval" badge

Once the Division Superintendents approve the app, remove the prototype badge:

```
# Edit the main page file
nano src/app/page.tsx

# Find and delete the "Prototype — For Approval" span
# (Search for "Prototype" — there are 2 occurrences: header and footer)

# Also remove it from PDF reports:
nano src/lib/pdfReport.ts
# (Search for "PROTOTYPE" and delete the header text)

# And from the printable questionnaire:
nano src/components/dashboard/PrintableQuestionnaire.tsx
# (Search for "Prototype" and delete the badge div)

# Rebuild and restart:
bun run build
pm2 restart rcsi-dashboard
```

5.2 Add the official El Salvador Division logo

To add the official logo:

```
# Upload the logo image to the public directory
scp logo.png root@YOUR_SERVER_IP:/var/www/rcsi-dashboard/public/logo.png

# Edit the layout to use the logo
nano src/app/layout.tsx
# Update the icons section to reference /logo.png

# Rebuild and restart
bun run build
pm2 restart rcsi-dashboard
```

5.3 Run clean slate for pilot deployment

Before launching for pilot testing, wipe the demo data so schools start with a clean database:

```
cd /var/www/rcsi-dashboard
bun run scripts/clean-slate.ts

# Verify the database is empty:
# The dashboard should show "0 schools" and empty charts
# Schools can now upload their own data via the Upload tab
```

5.4 Set up automatic database backups

Set up a daily backup of the SQLite database to prevent data loss:

```
# Create a backup script
cat > /var/www/rcsi-dashboard/backup.sh << 'EOF'
#!/bin/bash
BACKUP_DIR="/var/backups/rcsi-dashboard"
DATE=$(date +%Y-%m-%d_%H%M%S)
mkdir -p $BACKUP_DIR
cp /var/www/rcsi-dashboard/db/custom.db "$BACKUP_DIR/custom_$DATE.db"
# Keep only the last 30 days of backups
find $BACKUP_DIR -name "*.db" -mtime +30 -delete
EOF

# Make it executable
chmod +x /var/www/rcsi-dashboard/backup.sh

# Add a daily cron job (runs at 2:00 AM)
crontab -e
# Add this line:
0 2 * * * /var/www/rcsi-dashboard/backup.sh
```

Chapter 6 — Maintenance & Monitoring

This chapter covers routine maintenance tasks, monitoring the app’s health, and updating the application when new features are added.

6.1 Daily monitoring commands

Task	Command	When
Check app status	<code>pm2 status</code>	Daily
View recent logs	<code>pm2 logs rcsi-dashboard --lines 50</code>	Daily
Check disk space	<code>df -h</code>	Weekly
Check memory usage	<code>free -h</code>	Weekly
Check database size	<code>ls -lh /var/www/rcsi-dashboard/db/</code>	Monthly
List backups	<code>ls -lh /var/backups/rcsi-dashboard/</code>	Monthly

Table 6.1 — Routine maintenance commands.

6.2 Updating the application

When new features or fixes are made to the application, deploy them to the server:

```
# 1. Transfer updated files from your local computer
rsync -avz --exclude node_modules --exclude .next \
/path/to/rcsi-dashboard/ root@YOUR_SERVER_IP:/var/www/rcsi-dashboard/

# 2. On the server, install any new dependencies
cd /var/www/rcsi-dashboard
bun install

# 3. Rebuild
bun run build

# 4. Restart the app (zero downtime)
pm2 restart rcsi-dashboard

# The app is back online with the new features.
```

6.3 Restarting after a server reboot

If the server reboots (planned or unplanned), PM2 should automatically restart the app. Verify:

```
# After reboot, check if the app is running
pm2 status

# If it's "stopped" or "errored", start it manually:
pm2 restart rcsi-dashboard

# If PM2 itself didn't start on boot, run:
pm2 resurrect
```

6.4 Viewing the upload audit log

Every upload operation is logged in the database. View the audit log via the Archive tab in the dashboard, or query it directly:

```
# Install sqlite3 if not already installed
apt install -y sqlite3

# View recent uploads
sqlite3 /var/www/rcsi-dashboard/db/custom.db \
"SELECT timestamp, mode, fileName, rowCount, status FROM UploadBatch ORDER BY
timestamp DESC LIMIT 20;"
```

Chapter 7 — Troubleshooting

This chapter covers the most common issues and their solutions.

Problem	Cause	Solution
App not accessible (browser shows "Connection refused")	PM2 process not running or port 3000 not listening.	Run: <code>pm2 restart rcsi-dashboard</code> . Check: <code>pm2 logs rcsi-dashboard --lines 50</code>
App shows "0 schools" and empty charts	Database is empty (clean slate) or database path is wrong.	Check <code>.env</code> has <code>DATABASE_URL="file:./db/custom.db"</code> . Run: <code>bun run scripts/seed.ts</code> for demo data.
Upload fails with "Another upload in progress"	Another upload is currently running (concurrency mutex).	Wait 30 seconds and try again. The mutex releases automatically after the upload completes.
SSL certificate expired	Let's Encrypt certificate needs renewal (happens automatically with Caddy).	For Caddy: <code>systemctl restart caddy</code> . For nginx: <code>certbot renew</code>
Server disk full	Old backups or PM2 logs accumulating.	Run: <code>pm2 flush</code> (clear logs). Delete old backups: <code>find /var/backups -mtime +30 -delete</code>
App crashes after update	New dependencies not installed or build failed.	Run: <code>bun install && bun run build && pm2 restart rcsi-dashboard</code>
Database locked error	Two processes trying to write to SQLite simultaneously.	Restart PM2: <code>pm2 restart rcsi-dashboard</code> . The upload mutex should prevent this.

Table 7.1 — Common issues and solutions.

7.1 Getting help

If you encounter an issue not covered in this guide, collect the following information before contacting support:

- PM2 logs: `pm2 logs rcsi-dashboard --lines 100 > pm2-logs.txt`
- System info: `uname -a && free -h && df -h`
- App version: check the footer of the dashboard
- Browser console errors (F12 → Console tab)
- The exact error message or unexpected behavior

Appendix A — Quick Reference Card

A one-page summary of all essential commands. Print this and keep it near the server.

A.1 Essential Commands

Start the app:

```
pm2 start "bun run start" --name rcsi-dashboard
```

Stop the app:

```
pm2 stop rcsi-dashboard
```

Restart the app:

```
pm2 restart rcsi-dashboard
```

View logs:

```
pm2 logs rcsi-dashboard --lines 50
```

Check status:

```
pm2 status
```

Rebuild after code update:

```
cd /var/www/rcsi-dashboard
bun install && bun run build && pm2 restart rcsi-dashboard
```

Seed demo data:

```
cd /var/www/rcsi-dashboard
bun run scripts/seed.ts
```

Clean slate (wipe all data):

```
cd /var/www/rcsi-dashboard
bun run scripts/clean-slate.ts
```

Backup database:

```
cp /var/www/rcsi-dashboard/db/custom.db
/var/backups/rcsi-dashboard/manual_backup_$(date +%Y%m%d).db
```

Check SSL certificate:

```
# For Caddy:
systemctl status caddy

# For nginx:
certbot certificates
```

A.2 File Locations

Item	Path
Application directory	/var/www/rcsi-dashboard/
Database file	/var/www/rcsi-dashboard/db/custom.db

Item	Path
Environment file	/var/www/rcsi-dashboard/.env
PM2 logs	~/.pm2/logs/rcsi-dashboard-out.log
Caddy config	/etc/caddy/Caddyfile
nginx config	/etc/nginx/sites-available/rcsi-dashboard
Backups	/var/backups/rcsi-dashboard/

Table A.1 — File locations on the server.